ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук

Третьяков Александр Юрьевич

Реализация расширения для СУБД PostgreSQL

Финальная работа по направлению подготовки 09.04.04 «Программная инженерия»

образовательная программа «Продвинутый SQL»

Преподаватель: Моргунов Е.П.

Москва 2026

1. Введение

Расширение `loan_annuity` предназначено для расчёта аннуитетных платежей по кредиту, построения графика погашения, вычисления полной стоимости кредита (ПСК) и экспорта графика в формат JSON.

Расширение разработано для PostgreSQL 16 и использует язык PL/Python3. Оно может быть полезно для финансовых приложений, банковских систем, а также для учебных целей при изучении кредитных расчётов.

2. Состав расширения

Расширение включает следующие объекты базы данных:

2.1. Тип данных `payment_row`

Составной тип, описывающий одну строку графика платежей:

Поле	Тип	Описание
`payment_date`	date	Дата платежа
`mea`	numeric	Минимальный ежемесячный платёж (МЕП)
`interest_amount`	numeric	Сумма процентов в МЕП
`principal_amount`	numeric	Сумма основного долга в МЕП
`remaining_balance`	numeric	Остаток долга после платежа
`interest_rate`	numeric	Годовая процентная ставка

2.2. Таблица `payment_schedules`

Хранит все сгенерированные графики платежей:

Поле	Тип	Описание
`accountid`	integer	Идентификатор договора
`calcid`	integer	Номер расчёта (в рамках одного договора)
`calc_date`	date	Дата построения графика
`payment_date`	date	Дата платежа
`mea`	numeric	МЕП
`interest_amount`	numeric	Сумма процентов
`principal_amount`	numeric	Сумма основного долга
`remaining_balance`	numeric	Остаток после платежа
`interest_rate`	numeric	Процентная ставка

Первичный ключ: `(accountid, calcid, payment_date)`

2.3. Функции

`build_schedule(accountid integer, amount numeric, rate numeric, months integer, issue_date text)`

Строит график аннуитетных платежей, вставляет его в таблицу `payment_schedules` и возвращает результат в виде набора строк типа `payment_row`.

Параметры:

- `accountid` – идентификатор договора
- `amount` – сумма кредита
- `rate` – годовая процентная ставка (в %)
- `months` – срок кредита в месяцах
- `issue_date` – дата выдачи кредита (в формате YYYY-MM-DD)

Возвращает: `SETOF payment_row`

`calculate_psc(accountid integer, calcid integer)`

Вычисляет полную стоимость кредита (ПСК) по сохранённому графику.

Параметры:

- `accountid` – идентификатор договора
- `calcid` – номер расчёта

Возвращает: `numeric` – ПСК в процентах (например, 7.35)

`export_schedule_json(accountid integer, calcid integer)`

Экспортирует график платежей в формате JSON.

Параметры:

- `accountid` – идентификатор договора
- `calcid` – номер расчёта

Возвращает: `jsonb` – график в виде JSON-массива

3. Математические формулы

3.1. Аннуитетный платёж

Ежемесячный аннуитетный платёж рассчитывается по формуле:

$$ P = S \times \frac{i \times (1 + i)^n}{(1 + i)^n - 1} $$

где:

- P – ежемесячный платёж
- S – сумма кредита
- i – месячная процентная ставка, $i = \frac{\text{годовая ставка}}{12 \times 100}$
- n – срок кредита в месяцах

3.2. Полная стоимость кредита (ПСК)**

ПСК рассчитывается численным методом (метод Ньютона) из уравнения:

$$ S = \sum_{t=1}^{n} \frac{P_t}{(1 + \text{ПСК})^{t/12}} $$

где:

- S — сумма кредита
- P_t — платёж в периоде t
- n — количество платежей
- t — номер платежа (в месяцах)

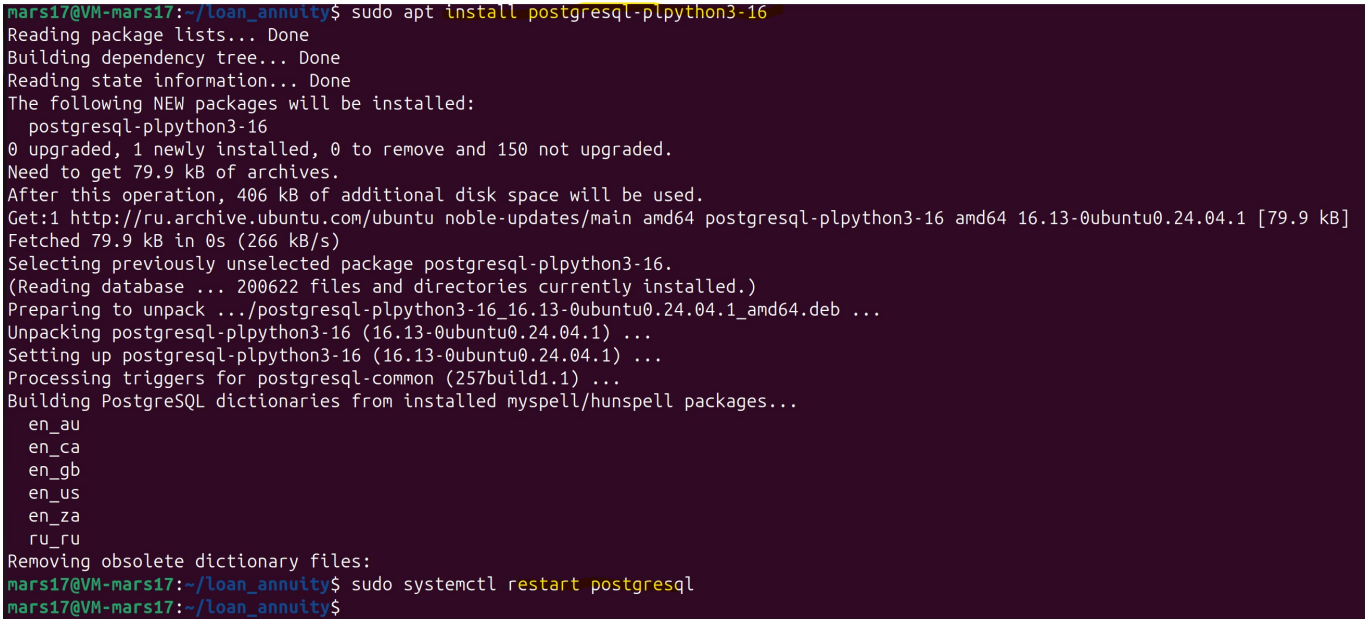
4. Установка расширения

4.1. Требования

- PostgreSQL 16
- PL/Python3 (пакет `postgresql-plpython3-16`)

4.2. Установка PL/Python3 (если не установлен)

```
sudo apt update
sudo apt install postgresql-plpython3-16
sudo systemctl restart postgresql
```



```
mars17@VM-mars17:~/loan_annuity$ sudo apt install postgresql-plpython3-16
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  postgresql-plpython3-16
0 upgraded, 1 newly installed, 0 to remove and 150 not upgraded.
Need to get 79.9 kB of archives.
After this operation, 406 kB of additional disk space will be used.
Get:1 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-plpython3-16 amd64 16.13-0ubuntu0.24.04.1 [79.9 kB]
Fetched 79.9 kB in 0s (266 kB/s)
Selecting previously unselected package postgresql-plpython3-16.
(Reading database ... 200622 files and directories currently installed.)
Preparing to unpack .../postgresql-plpython3-16_16.13-0ubuntu0.24.04.1_amd64.deb ...
Unpacking postgresql-plpython3-16 (16.13-0ubuntu0.24.04.1) ...
Setting up postgresql-plpython3-16 (16.13-0ubuntu0.24.04.1) ...
Processing triggers for postgresql-common (257build1.1) ...
Building PostgreSQL dictionaries from installed myspell/hunspell packages...
  en_au
  en_ca
  en_gb
  en_us
  en_za
  ru_ru
Removing obsolete dictionary files:
mars17@VM-mars17:~/loan_annuity$ sudo systemctl restart postgresql
mars17@VM-mars17:~/loan_annuity$
```

4.3. Установка расширения

1. Перейдите в каталог с исходными файлами расширения:

```
cd /loan_annuity
```

2. Выполните установку через Makefile:

```
sudo make install
```

3. Подключитесь к базе данных и создайте расширение:

```
CREATE EXTENSION loan_annuity;
```

```
mars17@VM-mars17:~/loan_annuity$ sudo make install
mkdir -p /usr/share/postgresql/16/extension
cp loan_annuity.control /usr/share/postgresql/16/extension/
cp sql/loan_annuity--1.0.sql /usr/share/postgresql/16/extension/
mars17@VM-mars17:~/loan_annuity$ sudo -u postgres psql
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# CREATE EXTENSION loan_annuity;
CREATE EXTENSION
postgres=#
```

4.4. Проверка установки

```
\dx loan_annuity
\d payment_schedules
\df build_schedule
```

```
postgres=# \dx loan_annuity
                                List of installed extensions
     Name     | Version | Schema |                      Description
--------------+---------+--------+-------------------------------------------------------
 loan_annuity | 1.0     | public | Расчёт аннуитетных платежей, графика погашения и ПСК
(1 row)

postgres=# \d payment_schedules
                Table "public.payment_schedules"
      Column       |  Type   | Collation | Nullable | Default
-------------------+---------+-----------+----------+---------
 accountid         | integer |           | not null |
 calcid            | integer |           | not null |
 calc_date         | date    |           |          |
 payment_date      | date    |           | not null |
 mea               | numeric |           |          |
 interest_amount   | numeric |           |          |
 principal_amount  | numeric |           |          |
 remaining_balance | numeric |           |          |
 interest_rate     | numeric |           |          |
Indexes:
    "payment_schedules_pkey" PRIMARY KEY, btree (accountid, calcid, payment_date)
```

```
postgres=# \df build_schedule
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                List of functions
 Schema |      Name      | Result data type  |                                                                                                                                                                                                                                                                                                                
```

```
postgres=# SELECT * FROM build_schedule(1, 1000000, 12, 12, '2024-01-01');
 payment_date |   mea    | interest_amount | principal_amount | remaining_balance | interest_rate
--------------+----------+-----------------+------------------+-------------------+---------------
 2024-01-31   | 88848.79 |        10000.00 |         78848.79 |         921151.21 |            12
 2024-03-01   | 88848.79 |         9211.51 |         79637.28 |         841513.93 |            12
 2024-03-31   | 88848.79 |         8415.14 |         80433.65 |         761080.29 |            12
 2024-04-30   | 88848.79 |         7610.80 |         81237.99 |         679842.30 |            12
 2024-05-30   | 88848.79 |         6798.42 |         82050.37 |         597791.93 |            12
 2024-06-29   | 88848.79 |         5977.92 |         82870.87 |         514921.06 |            12
 2024-07-29   | 88848.79 |         5149.21 |         83699.58 |         431221.49 |            12
 2024-08-28   | 88848.79 |         4312.21 |         84536.57 |         346684.91 |            12
 2024-09-27   | 88848.79 |         3466.85 |         85381.94 |         261302.97 |            12
 2024-10-27   | 88848.79 |         2613.03 |         86235.76 |         175067.21 |            12
 2024-11-26   | 88848.79 |         1750.67 |         87098.12 |          87969.10 |            12
 2024-12-26   | 88848.79 |          879.69 |         87969.10 |              0.00 |            12
(12 rows)

postgres=#
```

Пример 2. Расчёт ПСК для этого же кредита

```
SELECT calculate_psc(1, 1);
```

Результат: `12.68` (процентов)

```
postgres=# SELECT calculate_psc(1, 1);
 calculate_psc
---------------
         12.68
(1 row)

postgres=#
```

5.3. Экспорт графика в JSON

```
SELECT jsonb_pretty(export_schedule_json(1, 1));
```

```
               jsonb_pretty
----------------------------------------
 [                                      +
     {                                  +
         "mea": "88848.79",             +
         "payment_date": "2024-01-31",  +
         "interest_rate": "12",         +
         "interest_amount": "10000.00", +
         "principal_amount": "78848.79", +
         "remaining_balance": "921151.21"+
     },                                 +
     {                                  +
         "mea": "88848.79",             +
         "payment_date": "2024-03-01",  +
         "interest_rate": "12",         +
         "interest_amount": "9211.51",  +
         "principal_amount": "79637.28", +
         "remaining_balance": "841513.93"+
     },                                 +
     {                                  +
         "mea": "88848.79",             +
         "payment_date": "2024-03-31",  +
         "interest_rate": "12",         +
         "interest_amount": "8415.14",  +
         "principal_amount": "80433.65", +
         "remaining_balance": "761080.29"+
     },                                 +
     {                                  +
         "mea": "88848.79",             +
         "payment_date": "2024-04-30",  +
         "interest_rate": "12",         +
         "interest_amount": "7610.80",  +
         "principal_amount": "81237.99", +
         "remaining_balance": "679842.30"+
     },                                 +
     {                                  +
         "mea": "88848.79",             +
         "payment_date": "2024-05-30",  +
         "interest_rate": "12",         +
:
```

Результат: JSON-массив с полной информацией о каждом платеже.

6. Файлы расширения

В состав расширения входят следующие файлы:

- `loan_annuity.control` – управляющий файл
- `sql/loan_annuity--1.0.sql` – SQL-скрипт установки
- `Makefile` – файл для сборки (опционально)

7. Заключение

Разработанное расширение предоставляет удобный инструмент для расчёта аннуитетных кредитов непосредственно внутри базы данных PostgreSQL. Оно может быть использовано в банковских и финансовых системах, а также в учебных целях.